

Lab 1

Instructions for Lab Submission:

1. File Naming and Submission:
 - a. Submit your solution as a zip file named **lab1_<ERP>.zip**, where **<ERP>** is your ERP number. For example, if your ERP number is 12345, the file should be named *lab1_12345.zip*.
 - b. Ensure that the zip file contains:
 - i. C++ source files named as **task<number>.cpp** (e.g., **task1.cpp**).
 - ii. No unnecessary files (such as executables, temporary files, or editor configs).
 - c. Do not submit **TEXT(.txt)** files.
2. Code Organization:
 - a. Write each task in its own **.cpp** file (as specified above).
 - b. Ensure that your code is modular and logically separated into functions when specified.
3. Code Neatness:
 - a. Ensure your code is well-formatted and easy to read. Use proper indentation and meaningful variable names.
 - b. Include appropriate comments to explain the logic where necessary (especially for functions and complex sections of the code).
 - c. **Add clear explanations where required (or specified)** to make the logic of your program easy to follow.
4. Code Compilation and Functionality:
 - a. Ensure your code compiles without errors or warnings. You should be able to compile and run your code on any system with a standard C++ compiler (e.g., GCC, Clang, MSVC).
 - b. Double-check that the functionality of the program meets the requirements outlined in the task.
5. Additional Instructions:
 - a. Do not use any libraries or features that have not been covered in the course material, unless specifically instructed.
 - b. Make sure your code adheres to the principles of good software development (e.g., modular functions, minimal repetition, and no hardcoding).
 - c. If you encounter any issues or require clarifications, reach out before the submission deadline.
6. Deadline:
 - a. Ensure that you submit your lab on time, before the deadline. Late submissions will not be accepted.
 - b. Double-check that the zip file is correctly named and contains all necessary files before submitting.
7. Plagiarism Policy:
 - a. The work submitted must be entirely your own. Any code or content copied from others (including online sources, classmates or AI) will result in an automatic zero for the task.

Lecture Review

The Bare Minimum

A basic C++ program usually looks like this:

```
File: simple.cpp
1 // Preprocessor Directives
2
3 int main() {
4     // Code...
5 }
```

This is a completely valid program. If you try to run it, nothing will happen, but it will compile successfully. From this we learn an important point: the only thing the compiler really demands is the **main** function. Program execution always begins with **main**, and without it no program can compile.

Preprocessor Directives

Although the bare minimum compiles, it does not do anything useful. To make the program perform meaningful tasks, we must provide it with access to additional functionality. This is done by giving instructions to the preprocessor.

A **preprocessor directive** is a command that the compiler handles before the actual compilation begins. One of the most common is the **#include directive**, which instructs the compiler to copy the contents of another file into the program. By doing this, the compiler learns about new functions, objects, and definitions.

The line **#include <iostream>** tells the compiler to include the header file named **iostream**. When the compiler sees this directive, it *effectively* pastes the contents of **iostream** into the program. This header provides many of the most basic tools of C++, including the standard input and output streams. The angle brackets <> indicate that the header should be searched for in the *compiler's standard library directories*.

```
File: simple.cpp
1 #include <iostream>
2
3 int main() {
4     // Code...
5 }
```

Printing with std::cout

One of the most useful objects defined in **iostream** is **std::cout**. This object represents the standard output stream, which usually means the console window. To send data into this stream, we use the insertion operator <<.

If we extend our program to use it, the code becomes:

```
File: simple.cpp
1 #include <iostream>
2
3 int main() {
4     std::cout << "Hello, World!" << std::endl;
5 }
```

When compiled and run, this program displays the text "Hello, World!" on the screen. If we want the output to end with a newline, we can add either **\n** inside the string or attach **std::endl** at the end. For example:

```
std::cout << "Hello, World!" << std::endl;
```

The **std::endl** manipulator inserts a newline character and flushes the output buffer, making sure the text appears immediately.

Compiling and Running the Program

The process of writing, compiling, and running the program can be done directly from the command line, without the need for a special editor.

1. First, create a new folder on your **Desktop** and open it.
2. In **File Explorer's** address bar, type **cmd** and press Enter. This opens Command Prompt in the folder you are currently in. (*Current Directory, or cwd – Current working directory*)
3. Next, type the command:

notepad hello.cpp

This will create a new file named *hello.cpp* (if it doesn't exist) and open it in Notepad.

4. Write the code into this file and save it.
5. Back in Command Prompt, compile the file using:

g++ hello.cpp -o hello.exe

6. If the code is free of errors, the compiler will produce an executable named *hello.exe*. You can run it by typing:

hello.exe

If there is a mistake, such as forgetting a semicolon after a statement, the compiler will print an error message showing the line number and the nature of the problem.

Escape Sequences

When printing text, sometimes we need special characters that cannot be typed directly, such as newlines or quotation marks inside a string. For this purpose, C++ provides escape sequences. These are written as a backslash followed by another character.

- ***\n*** moves the cursor to a new line.
- ***\t*** inserts a tab.
- ***\r*** returns the cursor to the beginning of the same line.
- ***\b*** moves one character backward (backspace).
- ****** prints a backslash.
- ***\"*** prints a double quote.
- ***\'*** prints a single quote.

For example, the following code uses ***\n*** to print two lines of output:

```
std::cout << "Hello, World!\n";  
std::cout << "Welcome to C++ Programming\n";
```

The console output will be:

```
Hello, World!  
Welcome to C++ Programming
```

Lab Tasks - Extended

Task 7: Power!!!

Write a program that declares a variable of type integer and square it using the `<cmath>` library. The program should:

- Declare a variable of type integer
- Use the `pow(arg1, arg2)` from `<cmath>` to calculate the square
- The `pow(arg1, arg2)` takes in two arguments. `arg1` is the integer you want to apply the power to and `arg2` is the amount of power you want to apply.

```
Intro-To-Programming-CPP/Lab_Solutions/Lab01
> g++ T7E.cpp && ./a.out
Square of 5 is 25
```

Task 8: Calculating the Hypotenuse

Write a program that calculates the hypotenuse of a triangle. The program should:

- Declares two variables `a` and `b` for the lengths of the right triangle's sides.
- Calculates the hypotenuse using the formula:
- Print the result

$$c = \sqrt{a^2 + b^2}$$

Output Workflow:

1. Side `a` of right angle triangle
2. Side `b` of right angle triangle
3. Hypotenuse of the right triangle

```
Intro-To-Programming-CPP/Lab_Solutions/Lab01
> g++ T8E.cpp && ./a.out
Side a = 3
Side b = 4
Hypotenuse = 5
```

Task 9: Temperature Conversions

Write a program that converts temperatures from Fahrenheit to Celsius. The program should:

- Declare a variable for Celsius as an integer
- Declare a variable for Fahrenheit as an integer
- Formula for conversion:
- Display the Celsius integer and the converted one as well.

$$F = (C \times 9/5) + 32$$

Output Workflow:

1. Display Celsius integer
2. Display Fahrenheit integer

```
Intro-To-Programming-CPP/Lab_Solutions/Lab01
> g++ T9E.cpp && ./a.out
Celsius: 25
Fahrenheit: 77
```

Task 10: Going in Circles!

Write a program which calculates the area of the circle. The program should:

- Declare a radius variable
- Declare an area variable
- Calculate the area of a circle using this formula:
- Display the variables

$$\text{Area} = \pi \times r^2$$

Output Workflow:

1. Display the radius of the circle
2. Display the area of the circle

```
Intro-To-Programming-CPP/Lab_Solutions/Lab01  
> g++ T10E.cpp && ./a.out  
Radius: 5  
Area of Circle: 78
```